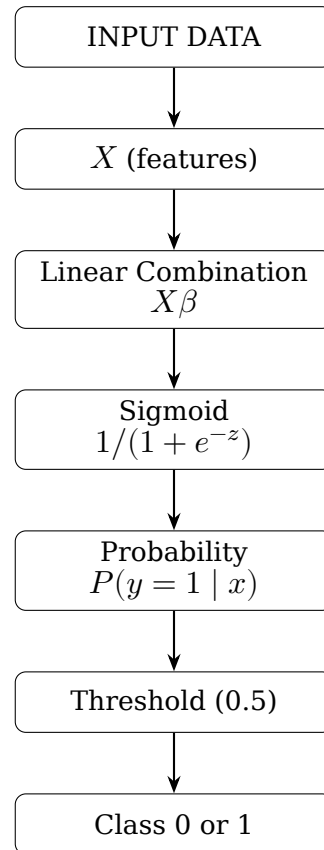


Logistic Regression — Full Derivation From Scratch

Contents

Overview Flowchart	3
Step 1 — Input Data	4
Step 2 — Design Matrix X	4
Step 3 — Linear Combination	4
Step 4 — The Sigmoid Function	5
Step 5 — Probability Interpretation	5
Step 6 — Why Not Reuse the Squared-Error Cost?	6
Step 7 — Deriving the Cost Function (Log-Loss / Cross-Entropy)	6
Step 8 — Differentiating the Cost Function	7
Step 9 — Why No Closed-Form Solution?	8
Step 10 — Gradient Descent Update Rule	8
Step 11 — Iteration 1 (fully worked)	8
Step 12 — Iteration 2 (fully worked)	9
Step 13 — Convergence Over Many Iterations	10
Step 14 — Final Hypothesis	11
Step 15 — Decision Boundary	11
Step 16 — Training Predictions, Threshold, and Classification	11
Step 17 — Prediction on New Input	11
Step 18 — Assumptions and Practical Notes	12
Step 19 — When to Use Logistic Regression	12

Overview Flowchart



Everything up through the linear combination $X\beta$ is **identical** to linear regression. What's new is squashing that number through the **sigmoid** to get a probability, then thresholding it into a class. This document derives every arrow above from scratch, on a concrete worked example.

Step 1 — Input Data

A binary classification dataset — one feature x , and a label $y \in \{0, 1\}$:

x	y
1	0
2	1
3	0
4	1

$n = 4$ observations. Note this data is **not** perfectly separable by any single threshold on x (class alternates 0, 1, 0, 1) — deliberately chosen so the model has to make trade-offs, just like real data usually does.

Terms: binary classification, label, linearly separable (its absence, here).

Step 2 — Design Matrix X

Same design matrix construction as linear regression — intercept column plus feature column:

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 1 \end{bmatrix}$$

Terms: design matrix.

Step 3 — Linear Combination

Exactly as in linear regression, first compute a raw linear score z :

$$z = X\beta = \beta_0 + \beta_1 x$$

This z can range over all real numbers $(-\infty, \infty)$ — but a probability must live in $[0, 1]$. That mismatch is exactly what the next step fixes.

Terms: linear combination, logit (another name for z , introduced formally in Step 5).

Step 4 — The Sigmoid Function

Define:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Why this specific function? It has exactly the properties needed to turn z into a probability:

- As $z \rightarrow +\infty$: $e^{-z} \rightarrow 0$, so $\sigma(z) \rightarrow \frac{1}{1+0} = 1$.
- As $z \rightarrow -\infty$: $e^{-z} \rightarrow \infty$, so $\sigma(z) \rightarrow \frac{1}{1+\infty} = 0$.
- At $z = 0$: $\sigma(0) = \frac{1}{1+e^0} = \frac{1}{1+1} = 0.5$.
- It is smooth and strictly increasing, so larger z always means a higher predicted probability.

Derivative of the sigmoid (needed later for the cost function's gradient). Using the quotient rule on $\sigma(z) = (1 + e^{-z})^{-1}$:

$$\sigma'(z) = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2}$$

Rewrite $e^{-z} = (1 + e^{-z}) - 1$:

$$\sigma'(z) = \frac{(1 + e^{-z}) - 1}{(1 + e^{-z})^2} = \frac{1}{1 + e^{-z}} - \frac{1}{(1 + e^{-z})^2} = \sigma(z) - \sigma(z)^2$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

This remarkably clean identity — the derivative expressed in terms of the function itself — is what makes the gradient in Step 8 simplify so nicely.

Terms: sigmoid/logistic function, quotient rule, self-referential derivative identity.

Step 5 — Probability Interpretation

Define the model's predicted probability that $y = 1$ given x :

$$\hat{y} = \sigma(z) = \sigma(\beta_0 + \beta_1 x) = P(y = 1 \mid x)$$

Since probabilities of the two classes must sum to 1:

$$P(y = 0 \mid x) = 1 - \hat{y}$$

The quantity $z = \beta_0 + \beta_1 x$ itself is called the **logit** — the log-odds of the positive class (a fact we'll use again in Step 7).

Terms: predicted probability, logit, log-odds.

Step 6 — Why Not Reuse the Squared-Error Cost?

If we plugged $\hat{y} = \sigma(z)$ into the same $J = \frac{1}{2n} \sum (y_i - \hat{y}_i)^2$ used for linear regression, the resulting cost surface would generally be **non-convex** in β (because of the sigmoid's curvature) — gradient descent could get stuck in a local minimum. Logistic regression instead uses a cost function derived from **maximum likelihood**, which stays convex.

Terms: non-convexity, local minimum, maximum likelihood.

Step 7 — Deriving the Cost Function (Log-Loss / Cross-Entropy)

Likelihood of one observation. Since $y_i \in \{0, 1\}$, the probability of observing the actual label y_i given the model's prediction $\hat{y}_i$ can be written compactly as:

$$P(y_i \mid x_i) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

(Check: if $y_i = 1$, this reduces to $\hat{y}_i$; if $y_i = 0$, this reduces to $1 - \hat{y}_i$ — exactly right in both cases.)

Likelihood of all n observations (assuming independence — just multiply the individual probabilities):

$$L(\beta) = \prod_{i=1}^n \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1-y_i}$$

Maximize the likelihood $\iff$ **minimize the negative log-likelihood** (logs turn the product into a sum, and flipping the sign turns “maximize” into “minimize” so it matches the cost-function convention from linear regression):

$$J(\beta) = -\frac{1}{n} \ln L(\beta) = -\frac{1}{n} \sum_{i=1}^n \left[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i) \right]$$

This is the **log-loss** (also called binary cross-entropy).

Terms: likelihood, log-likelihood, negative log-likelihood, log-loss, cross-entropy.

Step 8 — Differentiating the Cost Function

We need $\frac{\partial J}{\partial \beta_j}$. Focus on one term of the sum, $-[y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$, and apply the chain rule through $\hat{y} = \sigma(z)$ and $z = \beta_0 + \beta_1 x$.

Derivative with respect to $\hat{y}$:

$$\frac{\partial}{\partial \hat{y}} \left[-y \ln \hat{y} - (1 - y) \ln(1 - \hat{y}) \right] = -\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}}$$

Derivative of $\hat{y}$ with respect to z — this is exactly the sigmoid identity from Step 4:

$$\frac{\partial \hat{y}}{\partial z} = \sigma(z)(1 - \sigma(z)) = \hat{y}(1 - \hat{y})$$

Chain rule — multiply the two:

$$\frac{\partial}{\partial z} \left[-y \ln \hat{y} - (1 - y) \ln(1 - \hat{y}) \right] = \left(-\frac{y}{\hat{y}} + \frac{1 - y}{1 - \hat{y}} \right) \cdot \hat{y}(1 - \hat{y})$$

Distribute:

$$= -y(1 - \hat{y}) + (1 - y)\hat{y} = -y + y\hat{y} + \hat{y} - y\hat{y} = \hat{y} - y$$

This is the key simplification — **all the sigmoid machinery collapses down to just $(\hat{y} - y)$** .

Derivative of z with respect to β_j : since $z = \beta_0 + \beta_1 x$, we get $\frac{\partial z}{\partial \beta_0} = 1$ and

$$\frac{\partial z}{\partial \beta_1} = x.$$

Putting it together, averaged over all n points:

$$\boxed{\frac{\partial J}{\partial \beta_0} = \frac{1}{n} \sum (\hat{y}_i - y_i)}$$

$$\boxed{\frac{\partial J}{\partial \beta_1} = \frac{1}{n} \sum (\hat{y}_i - y_i) x_i}$$

Remarkable result: this has *exactly* the same form as the linear regression gradient from the earlier gradient-descent document — the only difference is that $\hat{y}_i$ is now $\sigma(\beta_0 + \beta_1 x_i)$ instead of $\beta_0 + \beta_1 x_i$ directly.

Terms: chain rule, gradient of log-loss.

Step 9 — Why No Closed-Form Solution?

Setting $\frac{\partial J}{\partial \beta} = 0$ gives $\frac{1}{n} \sum (\sigma(\beta_0 + \beta_1 x_i) - y_i) = 0$ (and similarly for β_1) — but β is now trapped **inside** a sigmoid, so this equation can't be rearranged algebraically into $\beta = (\dots)$ the way the linear normal equation could. There is no matrix-inversion shortcut here. This is precisely why logistic regression is fit with **gradient descent** (or related iterative optimizers like Newton's method / IRLS), never a normal equation.

Terms: transcendental equation, iterative optimization, Newton's method (mentioned for context, not derived).

Step 10 — Gradient Descent Update Rule

$$\beta_0 := \beta_0 - \alpha \frac{\partial J}{\partial \beta_0}, \quad \beta_1 := \beta_1 - \alpha \frac{\partial J}{\partial \beta_1}$$

Choices for this example: $\alpha = 0.5$, initialized at $\beta_0 = 0$, $\beta_1 = 0$.

Step 11 — Iteration 1 (fully worked)

Current parameters: $\beta_0 = 0$, $\beta_1 = 0$

Step A — linear scores $z_i = \beta_0 + \beta_1 x_i$: all zero, so $z = [0, 0, 0, 0]$

Step B — predicted probabilities $\hat{y}_i = \sigma(z_i) = \sigma(0) = 0.5$ for every point:

$$\hat{y} = [0.5, 0.5, 0.5, 0.5]$$

Step C — errors $\hat{y}_i - y_i$:

$$[0.5 - 0, 0.5 - 1, 0.5 - 0, 0.5 - 1] = [0.5, -0.5, 0.5, -0.5]$$

Step D — cost:

Using $J = -\frac{1}{n} \sum [y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$ with $\hat{y} = 0.5$ for all four points (so both $\ln \hat{y} = \ln(1 - \hat{y}) = \ln 0.5 \approx -0.6931$ regardless of y):

$$J = -\frac{1}{4} \times 4 \times \ln(0.5) = -\ln(0.5) \approx 0.6931$$

Step E — gradient for β_0 :

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{4}(0.5 - 0.5 + 0.5 - 0.5) = 0$$

Step F — gradient for β_1 :

$$\frac{\partial J}{\partial \beta_1} = \frac{1}{4}(0.5(1) - 0.5(2) + 0.5(3) - 0.5(4)) = \frac{1}{4}(0.5 - 1 + 1.5 - 2) = \frac{-1}{4} = -0.25$$

Update:

$$\beta_0 := 0 - 0.5(0) = 0 \qquad \beta_1 := 0 - 0.5(-0.25) = 0.125$$

Step 12 — Iteration 2 (fully worked)

Current parameters: $\beta_0 = 0, \beta_1 = 0.125$

Linear scores: $z = [0.125, 0.25, 0.375, 0.5]$

Predicted probabilities (evaluating the sigmoid at each z):

$$\hat{y}_1 = \sigma(0.125) \approx 0.5312, \quad \hat{y}_2 = \sigma(0.25) \approx 0.5622, \quad \hat{y}_3 = \sigma(0.375) \approx 0.5927, \quad \hat{y}_4 = \sigma(0.5) \approx 0.6225$$

Errors:

$$e_1 = 0.5312 - 0 = 0.5312, \quad e_2 = 0.5622 - 1 = -0.4378, \quad e_3 = 0.5927 - 0 = 0.5927, \quad e_4 = 0.6225 - 1 = -0.3775$$

Cost:

$$J \approx 0.6733$$

Gradient for β_0 :

$$\frac{\partial J}{\partial \beta_0} = \frac{1}{4}(0.5312 - 0.4378 + 0.5927 - 0.3775) = \frac{0.3086}{4} \approx 0.0772$$

Gradient for β_1 :

$$\begin{aligned} \frac{\partial J}{\partial \beta_1} &= \frac{1}{4}(0.5312(1) - 0.4378(2) + 0.5927(3) - 0.3775(4)) \\ &= \frac{1}{4}(0.5312 - 0.8756 + 1.7781 - 1.51) = \frac{-0.0763}{4} \approx -0.0191 \end{aligned}$$

Update:

$$\beta_0 := 0 - 0.5(0.0772) \approx -0.0386 \qquad \beta_1 := 0.125 - 0.5(-0.0191) \approx 0.1346$$

Step 13 — Convergence Over Many Iterations

Continuing the identical mechanical process (predictions $\rightarrow$ errors $\rightarrow$ cost $\rightarrow$ gradients $\rightarrow$ update):

Iteration	β_0	β_1	Cost J
1	0.0000	0.0000	0.69315
2	0.0000	0.1250	0.67643
3	\$-0.0386	0.1346	0.67332
5	\$-0.1114	0.1595	0.66744
10	\$-0.2820	0.2182	0.65453
50	\$-1.2081	0.5385	0.60494
200	\$-2.1382	0.8622	0.58713
1000	\$-2.2705	0.9082	0.58687
5000	\$-2.2705	0.9082	0.58687

By iteration $\approx$ \$1000, the parameters have essentially converged (no meaningful change afterward) — unlike linear regression’s exact one-shot answer, this convergence took many iterations, but it does settle to a stable value because (unlike a perfectly separable dataset) the cost function has a genuine finite minimum here.

Terms: convergence, plateauing.

Step 14 — Final Hypothesis

$$\hat{y} = \sigma(-2.2705 + 0.9082x) = \frac{1}{1 + e^{-(-2.2705 + 0.9082x)}}$$

Step 15 — Decision Boundary

The **decision boundary** is where the model is exactly 50/50 — i.e. where $z = 0$ (since $\sigma(0) = 0.5$):

$$-2.2705 + 0.9082x = 0 \implies x = \frac{2.2705}{0.9082} \approx \boxed{2.5}$$

Below $x = 2.5$, the model leans toward class 0; above it, class 1.

Terms: decision boundary.

Step 16 — Training Predictions, Threshold, and Classification

Apply the threshold rule: predict class 1 if $\hat{y} \geq 0.5$, else class 0.

x	y (actual)	$\hat{y} = \sigma(z)$	Predicted class	Correct?
1	0	0.2039	0	(OK)
2	1	0.3884	0	Wrong
3	0	0.6116	1	Wrong
4	1	0.7961	1	(OK)

Two of the four training points are misclassified — expected, since this dataset (Step 1) was deliberately **not linearly separable**: no single threshold on x can perfectly separate the alternating pattern 0, 1, 0, 1. The model finds the best compromise decision boundary ($x \approx 2.5$), not a perfect one.

Terms: threshold, classification, misclassification.

Step 17 — Prediction on New Input

New input: $x = 5$

$$z = -2.2705 + 0.9082(5) = -2.2705 + 4.541 = 2.2705$$

$$\hat{y} = \sigma(2.2705) = \frac{1}{1 + e^{-2.2705}} \approx \boxed{0.9064}$$

Since $0.9064 \geq 0.5$, the predicted class is $\boxed{1}$.

Also checking $x = 2.5$ (exactly on the decision boundary): $\hat{y} = \sigma(0) = 0.5$ exactly, so the model is maximally uncertain there — right at the 50/50 line, as expected.

Step 18 — Assumptions and Practical Notes

- **Linearity in the logit** — logistic regression assumes the *log-odds* (not y itself) is a linear function of the features.
 - **Independent observations** — same as linear regression.
 - **No severe multicollinearity** — same concern as multiple linear regression; unstable coefficients if features are near-duplicates.
 - **Large-sample behavior** — coefficient estimates and their standard errors rely on asymptotic theory; small datasets (like our $n = 4$ toy example) are for illustrating the mechanics, not for trusting the exact coefficient values statistically.
 - **Perfect separation is a real failure mode** — if the classes *were* perfectly separable by x (as in an earlier, discarded version of this example), gradient descent would push $|\beta| \rightarrow \infty$ forever, never truly converging — a well-known pathology of logistic regression, usually fixed with regularization (see Step 19).
-

Step 19 — When to Use Logistic Regression

Use it when:

- The target is **categorical with two classes** (or extended to multiple classes via one-vs-rest / multinomial logistic regression).
- You want **calibrated probabilities**, not just a hard classification — e.g. “73% chance of default,” not just “will default.”
- You want an **interpretable** model — each coefficient’s effect on the log-odds is directly readable, similar to linear regression’s coefficients.

Prefer alternatives when:

- Classes are **perfectly or near-perfectly separable** and you need finite, stable coefficients $\rightarrow$ add **regularization** (L2/ridge-style penalty is standard here, keeping the same gradient-descent machinery but adding a term that discourages $|\beta| \rightarrow \infty$).
- The **decision boundary is highly non-linear** in the raw features $\rightarrow$ add polynomial/interaction features (same idea as the polynomial regression document), or move to a fundamentally different model (decision trees, random forests, neural networks, SVM with a non-linear kernel).

- You have **more than two classes without a natural ordering** → multinomial logistic regression or another multi-class method.
- The relationship between features and the **log-odds** genuinely isn't linear even after feature engineering → non-linear classifiers become more attractive.

Summary Table

Order	Concept	Result
4	Sigmoid derivative	$\sigma'(z) = \sigma(z)(1 - \sigma(z))$
7	Cost function	Log-loss: $J = -\frac{1}{n} \sum [y \ln \hat{y} + (1 - y) \ln(1 - \hat{y})]$
8	Gradient	$\frac{\partial J}{\partial \beta_j} = \frac{1}{n} \sum (\hat{y}_i - y_i) x_{j,i}$ (same form as linear regression)
9	Closed form?	None — β trapped inside the sigmoid
13	Converged parameters	$\beta_0 \approx -2.2705, \beta_1 \approx 0.9082$
15	Decision boundary	$x \approx 2.5$
16	Training accuracy	2/4 correct (data not linearly separable)
17	Prediction at $x = 5$	$\hat{y} \approx 0.9064 \rightarrow$ class 1